

Inteligencia Artificial

Informe Final: Problema Multi Depot Vehicle Routing Problem with Time Windows

Pablo Retamales

26 de junio de 2026

Evaluación

Código Fuente (10 %):	_____
Resumen (2 %):	_____
Introducción (3 %):	_____
Representación (10 %):	_____
Descripción del algoritmo (20 %):	_____
Experimentos (10 %):	_____
Resultados (30 %):	_____
Conclusiones (10 %):	_____
Bibliografía (5 %):	_____
Nota Final (100):	_____

Resumen

El Multi-Depot Vehicle Routing Problem with Time Windows (MDVRP-TW) es una variante del Vehicle Routing Problem (VRP) que considera múltiples depósitos y restricciones de tiempo para la atención de clientes. El objetivo es determinar rutas de mínimo costo para una flota de vehículos que atiende clientes dentro de intervalos de tiempo definidos y retorna a su depósito de origen. Debido a su complejidad computacional, el MDVRP-TW ha sido abordado principalmente mediante metaheurísticas como búsqueda tabú y algoritmos genéticos, aunque también existen métodos exactos para instancias pequeñas. Este informe presenta una descripción general del problema, sus principales variantes, métodos de resolución y un modelo matemático de programación entera mixta.

1. Introducción

El presente informe tiene como propósito describir y analizar el Multi-Depot Vehicle Routing Problem with Time Windows (MDVRP-TW), un problema de optimización combinatoria de gran relevancia en el ámbito de la logística y la distribución. Este problema surge de la necesidad de coordinar flotas de vehículos que operan desde múltiples centros de distribución para atender a un conjunto de clientes con intervalos de tiempo definidos, buscando minimizar el costo total de las rutas.

En este informe se presenta la definición del problema, sus variables, parámetros, restricciones y objetivo, además de una revisión de los métodos utilizados para resolverlo. A través de este análisis, el informe busca consolidar el estado del arte del problema para identificar cuáles son las estrategias de resolución más eficientes y detectar las brechas metodológicas que

guiarán el desarrollo de trabajos futuros. Posteriormente, se expone su modelo matemático de programación entera mixta y se discuten tendencias actuales de investigación.

El interés por estudiar el MDVRP-TW se debe a su aplicación en sistemas reales de distribución y transporte, utilizados por empresas de logística como Amazon. El problema es de naturaleza NP-hard, ya que generaliza problemas de optimización combinatoria conocidos por ser NP-hard, como el problema del vendedor viajero (TSP) y sus variantes de ruteo de vehículos. Además, integra simultáneamente decisiones de asignación de clientes a depósitos, planificación de rutas y cumplimiento de ventanas de tiempo, lo que incrementa significativamente la complejidad del problema. La interacción entre estos subproblemas provoca un crecimiento combinatorio del espacio de búsqueda de soluciones, dificultando la obtención del óptimo global en tiempos computacionales razonables para instancias de gran tamaño. Por ello, el desarrollo de métodos capaces de encontrar soluciones de alta calidad en tiempos acotados sigue siendo un desafío relevante dentro del área de optimización combinatoria.

Para abordar este desafío, el acercamiento propuesto consiste en el diseño e implementación de la técnica de Simulated Annealing, enfocada en la exploración eficiente del espacio de búsqueda mediante múltiples operadores de vecindad. El comportamiento del algoritmo se evalúa bajo un entorno experimental estructurado sobre instancias desde 25 hasta 1000 clientes, permitiendo ajustar sus parámetros críticos. En cuanto a la estructura del documento, la Sección 2 detalla la definición general del problema; la Sección 3 revisa el estado del arte; la Sección 4 expone el modelo matemático formal; la Sección 5 describe la representación de soluciones; la Sección 6 profundiza en el algoritmo implementado; la Sección 7 detalla la metodología de la experimentación, la Sección 8 presenta los resultados de la experimentación y, finalmente, la Sección 9 presenta las conclusiones y trabajo futuro.

2. Definición del Problema

El Vehicle Routing Problem (VRP) es un problema de optimización combinatoria que consiste en determinar el conjunto de rutas de menor costo para una flota de vehículos que parte desde un depósito central para atender a un conjunto de clientes, retornando al mismo depósito [13]. El MDVRP-TW extiende esta formulación al considerar múltiples depósitos y ventanas de tiempo para la atención de los clientes.

Dado un grafo completo $G = (V, A)$, donde $V = D \cup C$ es el conjunto de todos los nodos, siendo D el conjunto de depósitos y C el conjunto de clientes, y A el conjunto de arcos (i, j) entre todos los pares de nodos, el objetivo es determinar un conjunto de rutas para una flota de vehículos tal que se minimice el costo total de distribución, que comprende la distancia recorrida, el tiempo empleado y las penalizaciones por incumplimiento de las ventanas de tiempo [6].

Cada ruta debe comenzar y terminar en el mismo depósito, cada cliente debe ser visitado exactamente una vez, la demanda acumulada (demanda total) de los clientes en cada ruta no puede superar la capacidad del vehículo, y cada cliente debe ser atendido dentro de su ventana de tiempo $[e_i, l_i]$.

El objetivo del problema, de forma explícita, es determinar un conjunto de rutas para la flota de vehículos que permita atender a todos los clientes respetando sus restricciones de capacidad y de ventanas de tiempo, de manera que el costo total de operación sea lo más bajo posible. Este costo incluye principalmente la distancia recorrida, el tiempo de desplazamiento y posibles penalizaciones asociadas al incumplimiento de los intervalos de atención de los clientes.

2.1. Variables de Decisión

Las principales decisiones que deben determinarse en el MDVRP-TW son:

- Determinar qué vehículo atenderá a cada cliente y el orden en que se realizarán las visitas.

- Establecer los tiempos de llegada e inicio de servicio en cada cliente.
- Calcular los posibles retrasos respecto a las ventanas de tiempo establecidas, los cuales pueden generar penalizaciones.

2.2. Restricciones

Las restricciones fundamentales del MDVRP-TW son las siguientes:

- Cada cliente debe ser atendido exactamente una vez.
- Todos los vehículos deben iniciar su recorrido desde el depósito que les ha sido asignado.
- La demanda total de los clientes atendidos en una ruta no puede superar la capacidad del vehículo.
- Los vehículos deben regresar a su depósito de origen dentro de los intervalos de tiempo establecidos; en caso contrario, se aplican las penalizaciones correspondientes.
- Si un vehículo llega antes de la apertura de la ventana de tiempo de un cliente, debe esperar hasta el inicio del servicio. Si llega después del límite establecido, se aplica una penalización proporcional al retraso.

2.3. Problemas Relacionados y Variantes

El MDVRP-TW se enmarca dentro de la familia de problemas VRP. Las variantes más relevantes incluyen:

- **Capacited Vehicle Routing Problem (CVRP)**: solo considera restricción de capacidad, sin ventanas de tiempo ni múltiples depósitos.
- **Vehicle Routing Problem with Time Windows (VRPTW)**: incorpora ventanas de tiempo, pero con un único depósito.
- **Multi-Depot Vehicle Routing Problem (MDVRP)**: considera múltiples depósitos sin restricciones de tiempo [15].
- **Multi-Depot Open Vehicle Routing Problem with Time Windows (MDOVRPTW)**: variante abierta, donde los vehículos no necesitan retornar al depósito de origen [10].
- **Multi-Depot Dynamic Vehicle Routing Problem with Time Windows (MD-DVRPTW)**: incorpora demanda dinámica de clientes en tiempo real [17].

3. Estado del Arte

El VRP fue introducido por Dantzig y Ramser en 1959 en el contexto de la distribución de gasolina desde un depósito central a múltiples estaciones [4]. La incorporación de ventanas de tiempo al VRP (VRPTW) fue formalizada y sistematizada por Solomon en 1987, quien propuso tanto heurísticas de construcción basadas en inserción como instancias de prueba que se han convertido en un estándar para evaluar algoritmos dentro del VRP [12]. La extensión al caso multi-depósito fue formalizada por Cordeau et al. en 1997 con el MDVRP, siendo extendida en 2001 al MDVRP-TW mediante una Búsqueda Tabú Unificada (Unified Tabu Search, UTS) [2].

3.1. Representación de Soluciones

Las representaciones de soluciones para el MDVRP-TW se clasifican principalmente en dos enfoques. La **representación directa** almacena explícitamente las rutas de cada vehículo y depósito, lo que facilita la evaluación de restricciones y la aplicación de operadores de búsqueda local [2]. Por su parte, la **representación indirecta** utiliza una permutación de clientes que posteriormente es transformada en rutas factibles mediante un procedimiento de decodificación, como *Split* [14]. Este último enfoque es frecuente en algoritmos genéticos debido a que favorece la generación de descendencia factible.

3.2. Métodos Exactos

Los métodos exactos para el MDVRP-TW, tales como Branch-and-Bound, Branch-and-Cut y generación de columnas, permiten obtener soluciones óptimas garantizadas. Sin embargo, su aplicabilidad práctica se limita a instancias de pequeño tamaño debido al crecimiento exponencial del espacio de búsqueda. La naturaleza NP-hard del problema hace que estos métodos sean inviables para instancias grandes con decenas de depósitos y cientos de clientes [13].

3.3. Búsqueda Tabú (TS)

La Búsqueda Tabú (Tabu Search, TS) ha sido históricamente uno de los métodos más efectivos para resolver el MDVRP-TW. Cordeau et al. [2] propusieron una heurística tabú unificada que maneja simultáneamente el VRPTW, el MDVRP-TW y el PVRPTW. El método utiliza movimientos de reubicación de clientes entre rutas y una función objetivo penalizada que permite aceptar soluciones infactibles de manera temporal. Este enfoque, basado en el manejo de restricciones blandas mediante penalización, facilita la exploración de regiones del espacio de búsqueda que de otro modo serían difíciles de alcanzar. Posteriormente, Cordeau y Maischberger [3] mejoraron este enfoque mediante búsqueda tabú iterada en paralelo, aprovechando arquitecturas de múltiples núcleos para reducir tiempos de cómputo.

Desde una perspectiva empírica, este enfoque demostró que el rendimiento de la búsqueda está estrechamente ligado al número de iteraciones fijadas. Para el MDVRP-TW, una ejecución estándar de 10^5 iteraciones permite alcanzar soluciones de alta calidad con un gap promedio inferior al 1,5% respecto a las mejores soluciones conocidas (Best-known solutions, BKS), requiriendo tiempos de cómputo de entre 5 minutos y un poco más de 3 horas dependiendo de la dimensión de la instancia. Asimismo, el algoritmo demuestra ser altamente eficiente para configuraciones rápidas, logrando soluciones factibles con un gap promedio por debajo del 5% en menos de 20 minutos al limitar la búsqueda a solo 10^4 iteraciones [3].

3.4. Búsqueda por Vecindad Variable (VNS)

La Búsqueda por Vecindad Variable (Variable Neighborhood Search, VNS) fue aplicada al MDVRP-TW por Polacek et al. [9], obteniendo resultados comparables y competitivos con la Búsqueda Tabú tradicional. El método explora vecindarios de tamaño creciente a partir de la solución actual, lo que permite escapar de óptimos locales mediante cambios sistemáticos de vecindario. Sadati et al. [10] propusieron una variante denominada Variable Tabu Neighborhood Search (VTNS), que incorpora elementos de búsqueda tabú durante la fase de perturbación del VNS. Desde una perspectiva empírica, al evaluar esta metodología sobre las 20 instancias de referencia de Cordeau, el algoritmo VTNS demostró una alta precisión al alcanzar un gap promedio de 0,05% en sus mejores soluciones respecto a las BKS vigentes en su momento, logrando igualar el récord histórico en 11 problemas y establecer 1 nueva cota superior [10].

3.5. Algoritmos Genéticos e Híbridos

Los algoritmos genéticos (AG) han sido ampliamente aplicados al MDVRP-TW. Vidal et al. [14] propusieron un algoritmo genético híbrido con manejo adaptativo de diversidad para una clase amplia de problemas VRP con ventanas de tiempo, obteniendo resultados de alta calidad gracias a la combinación de operadores de cruce ricos en información con búsqueda local intensiva.

Desde una perspectiva empírica, el análisis textual de los autores detalla que el *Hybrid Genetic Search with Advanced Diversity Control* (HGS-ADC) superó a todos los enfoques existentes del estado del arte en su momento, reportando que la inclusión de la proximidad temporal redujo el gap promedio de desviación de 0,68 % a 0,53 % frente a búsquedas puramente espaciales. Asimismo, las pruebas estadísticas de Wilcoxon ($p = 0,000$) y Levene ($p = 0,011$) confirmaron con alta significancia una mejora crítica en la estabilidad de las soluciones y una escalabilidad capaz de resolver instancias masivas de hasta 1000 clientes [14].

En trabajos recientes han explorado hibridaciones con búsqueda por gran vecindario. El Hybrid Adaptive Large Neighborhood Search (HALNS) de Voigt et al. [16] combina la capacidad exploratoria del ALNS con la recombinación de soluciones propia de los algoritmos genéticos, mostrando buen desempeño en instancias de referencia del MDVRP y problemas relacionados.

3.6. Búsqueda por Gran Vecindario Adaptativa (ALNS)

La Búsqueda por Gran Vecindario Adaptativa (Adaptive Large Neighborhood Search, ALNS), introducida por Ropke y Pisinger (2006), ha sido ampliamente utilizada en la resolución de variantes del VRP, incluyendo el MDVRP-TW. El ALNS alterna de manera adaptativa entre múltiples operadores de destrucción y reparación, asignando mayor peso a aquellos que han mostrado mejor desempeño histórico durante la búsqueda [7]. Wang et al. [17] aplicaron una variante del ALNS al problema dinámico multi-depósito con ventanas de tiempo (MD-DVRPTW), incorporando operadores de eliminación innovadores y un método de inserción basado en compatibilidad de ventanas de tiempo. Desde una perspectiva empírica, el ALNS ha demostrado una alta efectividad en problemas con restricciones temporales, logrando identificar históricamente 48 nuevas soluciones de referencia BKS. Esto valida la capacidad de su estructura adaptativa para manejar la complejidad en escenarios que, como la variante de Wang et al. [17], escalan con éxito hasta los 288 clientes mediante un mecanismo de inserción en tiempo constante $O(1)$.

3.7. Aceleración mediante GPU

Una tendencia reciente en la resolución del MDVRP-TW es el uso de unidades de procesamiento gráfico (GPU) para acelerar la evaluación de vecindarios dentro de metaheurísticas. Este enfoque permite explotar el paralelismo masivo de las GPU para reducir significativamente los tiempos de cómputo en instancias de gran escala.

Trabajos recientes han propuesto esquemas de aceleración tensorial basados en GPU para problemas de ruteo multi-depósito, integrando evaluaciones eficientes de movimientos y estrategias de actualización multi-movimiento tipo *leader-follower* (leader-follower strategy). Estas técnicas permiten explorar simultáneamente múltiples vecindarios, mejorando tanto la eficiencia computacional como la calidad de las soluciones obtenidas.

En este contexto, se han reportado mejoras significativas en instancias de referencia del MDVRP-TW mediante el algoritmo híbrido multifactorial por descomposición (MDFIHA), incluyendo la obtención de nuevas cotas superiores en problemas de pequeña y gran escala, evidenciando el potencial de estas técnicas para acelerar metaheurísticas modernas sin degradar el desempeño de la búsqueda [5].

Específicamente, en las instancias clásicas C01, la variante MDFIHA logra un gap promedio de -0.03 % y establece 4 nuevas cotas superiores (BKS). En las instancias de gran escala V13, a

pesar de emplear un límite de tiempo menor (7,200 s frente a los 12,960 s de enfoques en CPU), MDFIHA alcanza un gap de -0.22 % mejorando 19 BKS, mientras que su variante acelerada por GPU con algoritmos genéticos (MDFIHA-ETGA) potencia este rendimiento con un gap de -0.60 % y 26 nuevas soluciones récord. Se comprobó además que estas mejoras en la calidad de las soluciones y tiempos de cómputo son estadísticamente significativas frente a los algoritmos de referencia ($p < 0,05$) [5].

3.8. Variantes Modernas y Tendencias

La investigación reciente ha extendido el MDVRP-TW hacia problemas con mayor realismo:

- **Ventanas de tiempo blandas:** permiten relajar restricciones de tiempo mediante penalizaciones [6].
- **MDVRP-TW con emisión de carbono:** incorpora costos de emisión y cuotas de comercio de carbono en la función objetivo [11].
- **MDVRP-TW con demanda dinámica:** los clientes pueden realizar pedidos en tiempo real durante la ejecución de las rutas [17].
- **MDVRP-TW con depósito flexible:** los vehículos pueden retornar a un depósito distinto del de partida, reduciendo la distancia total recorrida [8].

La tendencia actual se orienta hacia métodos híbridos que combinan búsqueda local intensiva con estrategias para mantener la diversidad de soluciones. Además, se han propuesto enfoques basados en aprendizaje automático para predecir configuraciones de parámetros adecuadas en metaheurísticas aplicadas a problemas de ruteo [1].

4. Modelo Matemático

A continuación se presenta un modelo de programación entera mixta (MIP) para el MDVRP-TW, basado en la formulación de Cordeau et al. [2] y extendido según Li y He [6].

4.1. Conjuntos, Parámetros y Variables

Sea $V = D \cup C$ el conjunto de todos los nodos, con $D = \{n + 1, \dots, n + m\}$ los depósitos y $C = \{1, \dots, n\}$ los clientes. Se define K como el conjunto de todos los vehículos, donde cada vehículo $k \in K$ está asociado a un depósito de origen $o(k) \in D$.

Variables de decisión:

- $x_{ijk} \in \{0, 1\}$: toma el valor 1 si el vehículo k recorre el arco (i, j) , 0 en caso contrario.
- $t_{ik} \geq 0$: tiempo de inicio de servicio del vehículo k en el nodo i .
- $w_{ik} \geq 0$: penalización por violación de ventana de tiempo del vehículo k en el nodo i .

Parámetros del problema:

- $D = \{d_1, d_2, \dots, d_m\}$: conjunto de depósitos, con m depósitos en total.
- $C = \{c_1, c_2, \dots, c_n\}$: conjunto de clientes, con n clientes en total.
- k : número máximo de vehículos disponibles por depósito.
- Q : capacidad máxima de carga de cada vehículo.
- q_i : demanda del cliente $i \in C$.

- $[e_i, l_i]$: ventana de tiempo válida en la que puede ser atendido un cliente i .
- s_i : tiempo de servicio del cliente i .
- d_{ij} : distancia euclidiana entre los nodos i y j .
- v : velocidad constante de los vehículos.
- α : coeficiente de penalización por minuto fuera de la ventana de tiempo.
- M : constante suficientemente grande utilizada en las restricciones temporales (Big-M).

4.2. Función Objetivo

Se minimiza el costo total, compuesto por distancia recorrida y penalizaciones por incumplimiento de ventanas de tiempo:

$$\min \sum_{k \in K} \sum_{i \in V} \sum_{j \in V} d_{ij} \cdot x_{ijk} + \alpha \sum_{k \in K} \sum_{i \in C} w_{ik} \quad (1)$$

donde d_{ij} es la distancia entre los nodos i y j , y α es el coeficiente de penalización por minuto fuera de la ventana de tiempo.

El primer término minimiza la distancia total recorrida por la flota, mientras que el segundo incorpora penalizaciones asociadas al incumplimiento de las ventanas de tiempo.

4.3. Restricciones

(1) Cada cliente es visitado exactamente una vez: Esta restricción asegura que cada cliente sea atendido por un único vehículo proveniente de un único depósito, garantizando que se satisfaga la totalidad de la demanda sin duplicidad de visitas.

$$\sum_{k \in K} \sum_{j \in V} x_{ijk} = 1 \quad \forall i \in C \quad (2)$$

(2) Conservación de flujo en cada nodo: Establece la continuidad de las rutas e indica que si un vehículo k visita a un cliente i , tiene la obligación estricta de salir de dicho nodo hacia cualquier otro destino j , impidiendo rutas inconclusas.

$$\sum_{j \in V} x_{ijk} - \sum_{j \in V} x_{jik} = 0 \quad \forall i \in C, \forall k \in K \quad (3)$$

(3) Cada vehículo parte de su depósito de origen: Determina las condiciones de inicio de las rutas, indicando que cada vehículo k puede salir a lo sumo una vez desde el depósito que tiene asignado como origen, denotado por $o(k)$.

$$\sum_{j \in C} x_{o(k),j,k} \leq 1 \quad \forall k \in K \quad (4)$$

(4) Restricción de capacidad: Restringe la carga total transportada, obligando a que la suma de las demandas individuales q_i de los clientes asignados en la secuencia de recorrido de un vehículo k no exceda la capacidad máxima Q de la flota.

$$\sum_{i \in C} q_i \sum_{j \in V} x_{ijk} \leq Q \quad \forall k \in K \quad (5)$$

(5) Consistencia temporal (propagación de tiempos): Modela de forma lineal la secuencia del cronograma. Si el vehículo k transita directamente desde i hacia j ($x_{ijk} = 1$), el

tiempo de llegada a j (t_{jk}) debe ser mayor o igual al arribo en i (t_{ik}), más su respectivo tiempo de servicio s_i y el tiempo de viaje determinado por la distancia d_{ij} y la velocidad v . Si $x_{ijk} = 0$, la ecuación queda desactivada mediante el valor M .

$$t_{ik} + s_i + \frac{d_{ij}}{v} \leq t_{jk} + M(1 - x_{ijk}) \quad \forall i, j \in V, \forall k \in K \quad (6)$$

donde M es un valor suficientemente grande (constante de Big-M).

(6) Penalización por violación de la ventana de tiempo: Estas ecuaciones definen la variable de holgura no negativa w_{ik} , la cual cuantifica el retraso temporal incurrido por el vehículo k en caso de llegar después del límite superior permitido (l_i) por el cliente i .

$$w_{ik} \geq t_{ik} - l_i \quad \forall i \in C, \forall k \in K \quad (7)$$

$$w_{ik} \geq 0 \quad \forall i \in C, \forall k \in K \quad (8)$$

(7) Espera por ventana de tiempo temprana: Obliga a que si un vehículo arriba a un nodo antes de la hora de apertura permitida (e_i), el inicio del servicio se retrase hasta que la ventana horaria se active formalmente. El cálculo solo tiene efecto si el nodo i es visitado.

$$t_{ik} \geq e_i \cdot \sum_{j \in V} x_{jik} \quad \forall i \in C, \forall k \in K \quad (9)$$

(8) Naturaleza de las variables: Establece el dominio matemático del modelo, restringiendo las variables de ruteo x_{ijk} a valores binarios y los tiempos continuos de llegada t_{ik} a valores no negativos.

$$x_{ijk} \in \{0, 1\} \quad \forall i, j \in V, \forall k \in K \quad (10)$$

$$t_{ik} \geq 0 \quad \forall i \in V, \forall k \in K \quad (11)$$

4.4. Espacio de Búsqueda

El espacio de búsqueda del modelo anterior está determinado principalmente por las variables binarias x_{ijk} . Con n clientes, m depósitos y $|K|$ vehículos, el número de variables binarias es del orden de $O((n + m)^2 \cdot |K|)$, lo que para instancias medianas (e.g., $n = 100$, $m = 5$, $|K| = 20$) representa del orden de 10^5 variables binarias. Dado que el problema es NP-hard, el espacio de soluciones factibles crece de manera superpolinomial con el tamaño de la instancia, justificando el uso de metaheurísticas para instancias de mediano y gran tamaño.

5. Representación

Para abordar el MDVRP-TW mediante la técnica de Simulated Annealing, se define una representación estructurada basada en un vector de rutas activas. Cada componente de este vector corresponde a un objeto que modela de forma independiente la trayectoria de un vehículo, el cual está compuesto por un identificador numérico del depósito que especifica su origen y retorno, y una secuencia indexada de clientes que almacena de manera ordenada los índices de los nodos visitados. Por ejemplo, en un escenario con dos depósitos, la solución se estructura como un vector principal cuyos elementos corresponden a Ruta 0: [depot = 1, clientes = [3, 5, 1]] y Ruta 1: [depot = 2, clientes = [4, 2]].

Como se ilustra en la Figura 1, esta abstracción desacopla las trayectorias físicas en componentes lineales independientes indexados de forma contigua en memoria, facilitando tanto el acceso directo como las operaciones locales de mutación.

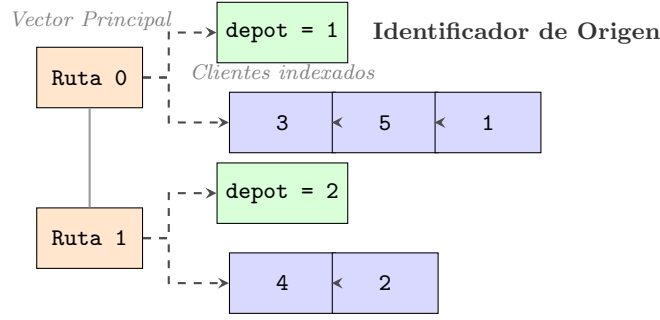


Figura 1: Esquema de la representación indexada de soluciones en memoria dinámica.

Esta representación ofrece ventajas críticas tanto para el modelado del problema como para el desempeño del algoritmo. En primer lugar, la estructura garantiza por diseño la factibilidad básica de las soluciones, asegurando que el espacio de búsqueda contenga la solución óptima sin desperdiciar cómputo en configuraciones inválidas fuera del dominio. Esto se logra al encapsular de forma fija el depósito de origen y retorno dentro de cada vehículo, lo que automatiza la validación del cierre de las rutas y permite calcular distancias euclidianas directamente, sin requerir mapeos externos.

En segundo lugar, al agrupar a los clientes secuencialmente, la verificación de capacidad y la propagación de las ventanas de tiempo $[e_i, l_i]$ se ejecutan mediante un recorrido lineal simple, lo que optimiza drásticamente el tiempo de cómputo por iteración. Finalmente, esta organización agiliza la aplicación de operadores de vecindad, ya que movimientos locales como transferencias o intercambios modifican exclusivamente los vectores internos de las rutas seleccionadas sin alterar el resto del sistema.

6. Descripción del algoritmo

La resolución del MDVRP-TW se aborda mediante una implementación de Simulated Annealing (SA). La propuesta se compone de tres etapas principales: generación de una solución inicial, exploración mediante operadores de vecindad y evaluación de soluciones mediante una función objetivo penalizada.

La búsqueda comienza con la construcción de una solución inicial ($S_{inicial}$), donde la lista de clientes se ordena aleatoriamente y se asigna a rutas respetando la capacidad de carga (Q). Cuando un cliente no puede ser incorporado a una ruta existente, se crea una nueva ruta desde el depósito más cercano. El procedimiento se resume en el siguiente pseudocódigo:

```

Procedimiento ConstruirSolucionInicial(instancia)
  MezclarAleatoriamente(clientes)

  Para cada cliente hacer
    Intentar asignarlo a una ruta existente

    Si no fue asignado Entonces
      Seleccionar depósito más cercano
      Crear nueva ruta
      Insertar cliente
    Fin Si
  Fin Para

  Retornar solución
Fin Procedimiento

```

La aleatorización del orden de los clientes permite generar soluciones iniciales diversas entre distintas ejecuciones, mientras que la selección del depósito más cercano busca reducir la distancia inicial de las rutas.

La calidad de cada solución se evalúa mediante la función de evaluación:

$$f(S) = \sum_{r \in S} D(r) + \alpha \cdot P_{tiempo}$$

donde $D(r)$ corresponde a la distancia total recorrida y P_{tiempo} representa la tardanza acumulada respecto a las ventanas de tiempo. La incorporación de esta penalización permite tratar las ventanas de tiempo como restricciones blandas, permitiendo explorar soluciones temporalmente infactibles durante la búsqueda. El parámetro α controla el equilibrio entre minimizar la distancia y respetar las restricciones temporales: valores altos favorecen la factibilidad, mientras que valores bajos aumentan la exploración.

El algoritmo ejecuta un ciclo de *MaxIteraciones*. En cada iteración se selecciona un operador de vecindad según las siguientes probabilidades:

- **Swap** (70 %): intercambia dos clientes dentro de una misma ruta.
- **Relocate** (20 %): traslada un cliente entre rutas.
- **Split** (10 %): divide una ruta en dos rutas asociadas al mismo depósito.

La distribución de probabilidades prioriza la intensificación mediante modificaciones locales frecuentes (*Swap*), mientras que *Relocate* y *Split* favorecen la diversificación al introducir cambios estructurales que permiten escapar de óptimos locales.

Al generar un vecino mediante los operadores, cualquier conflicto con las restricciones se registra y evalúa de forma posterior a su modificación. Específicamente, tras aplicar un movimiento, se verifica la restricción de capacidad (Q), descartando inmediatamente la solución en el flujo principal si esta se incumple. Para los vecinos factibles que superan esta validación, se calcula la diferencia de evaluación $\Delta = f(S_{vecino}) - f(S)$: las mejoras se aceptan de forma directa, mientras que las soluciones de peor calidad se aceptan con probabilidad $e^{-\Delta/T}$.

La mejor solución encontrada (S_{best}) se actualiza cada vez que se obtiene un menor costo. Paralelamente, la temperatura disminuye de forma geométrica cada *IntervaloEnfriamiento* iteraciones. Si se alcanza el límite de *MaxEstancamiento*, se aplica un recalentamiento que restablece la temperatura al 30 % de $T_{inicial}$ para favorecer la salida de óptimos locales.

Los parámetros $T_{inicial}$, *IntervaloEnfriamiento*, *MaxIteraciones* y *MaxEstancamiento* controlan respectivamente el nivel de exploración inicial, la velocidad de enfriamiento, la duración de la búsqueda y la activación del mecanismo de recalentamiento. El procedimiento general se resume en el siguiente pseudocódigo:

```
Procedimiento SimulatedAnnealing(instancia)
  S_inicial <- ConstruirSolucionInicialAleatoria(instancia)
  S <- S_inicial
  S_best <- S_inicial
  T <- T_inicial
  estancamiento <- 0

  Para iter <- 1 hasta MaxIteraciones Hacer
    S_vecino <- GenerarVecino(S) // Swap (70%), Relocate (20%) o Split (10%)

    Si No EsFactibleCapacidad(S_vecino, instancia) entonces
      Siguierte iteración
    Fin Si

    Delta <- CalcularEvaluacion(S_vecino) - CalcularEvaluacion(S)
```

```

Si (Delta < 0) O (GenerarAleatorioReal(0, 1) < e^(-Delta / T)) entonces
  S <- S_vecino
  Si CalcularEvaluacion(S) < CalcularEvaluacion(S_best) entonces
    S_best <- S
    estancamiento <- 0
  Fin Si
Sino
  estancamiento <- estancamiento + 1
Fin Si

Si (iter es múltiplo de IntervaloEnfriamiento) entonces
  T <- T * TasaEnfriamiento
Fin Si

Si estancamiento > MaxEstancamiento entonces
  T <- T_inicial * 0.3
  estancamiento <- 0
Fin Si
Fin Para

Retornar S_best
Fin Procedimiento

```

7. Experimentos

La metodología experimental evalúa el comportamiento del algoritmo Simulated Annealing en el MDVRP-TW y su sensibilidad frente a distintos parámetros de control. Al tratarse de una técnica estocástica, el desempeño del algoritmo depende de la aleatoriedad presente en la generación de soluciones iniciales y en la exploración del espacio de búsqueda.

Se consideran cinco instancias (C101, C101, C1.2.1, C1.4.1 y C110.1) con 25, 100, 200, 400 y 1000 clientes respectivamente, permitiendo analizar el comportamiento del algoritmo bajo distintos niveles de escala y complejidad asociados a las restricciones de capacidad y ventanas de tiempo.

Para reducir el efecto de la variabilidad inherente al algoritmo, cada configuración experimental se ejecuta utilizando cinco semillas aleatorias distintas. Los resultados reportados corresponden a los valores obtenidos en dichas ejecuciones, permitiendo evaluar la estabilidad y robustez de cada configuración de parámetros.

Durante la calibración del parámetro α , se utilizó una semilla fija por instancia con el fin de garantizar comparabilidad entre las distintas configuraciones evaluadas. En las pruebas finales de convergencia y robustez se emplearon cinco semillas aleatorias independientes por instancia.

Todas las ejecuciones se realizan en un entorno reproducible mediante archivos **Makefile**. Como indicadores de calidad se consideran el valor de la función objetivo, la distancia total recorrida, la cantidad de nodos penalizados y la penalización acumulada por ventanas de tiempo. El criterio de término corresponde a un máximo de 1.000.000 iteraciones.

El proceso de ajuste de parámetros se estructura en cuatro etapas:

1. **Calibración del parámetro α :** se evalúa el conjunto $\{0, 0,25, 0,5, 0,75, 1, 3, 5, 10\}$ sobre todas las instancias.
2. **Temperatura inicial (T_0):** se calibra este parámetro utilizando la instancia de 400 clientes, manteniendo fijos los demás parámetros del esquema de enfriamiento.
3. **Tasa de enfriamiento:** se ajusta el parámetro COOLING_RATE en el conjunto $\{0,95, 0,97, 0,98, 0,99, 0,995\}$, evaluando su impacto en la convergencia del algoritmo.
4. **Umbral de estancamiento:** se calibra MAX_STAGNATION en el rango de 1000 a 30000 iteraciones, evaluando su efecto en la capacidad del algoritmo para escapar de óptimos locales.

El entorno experimental se ejecuta en un equipo con procesador AMD Ryzen 5 6600H de 6 núcleos y 12 hilos, 16 GB de memoria RAM y sistema operativo Linux Fedora. La implementación no utiliza aceleración por GPU, ya que el algoritmo fue diseñado y ejecutado de forma secuencial sobre CPU.

8. Resultados

La presente sección resume los resultados obtenidos durante el ajuste de parámetros y la evaluación final del algoritmo.

8.1. Análisis del parámetro 'alpha' y escalabilidad

La primera fase experimental consistió en evaluar la sensibilidad del algoritmo frente a variaciones del parámetro α , el cual pondera los componentes de la función objetivo. Los datos consolidados para las cinco instancias de prueba se presentan en el Cuadro 1.

Cuadro 1: Resultados de la evaluación del parámetro α en las cinco instancias basales.

Instancia	Métrica	0	0,25	0,5	0,75	1	3	5	10
(25) C101 SEED=16	FO	145,201	162,577	162,577	233,531	162,577	162,577	283,036	213,249
	N. Penaliz.	19	0	0	0	0	0	0	0
	P. Tiempo	9767,63	0	0	0	0	0	0	0
	Distancia	145,201	162,577	162,577	233,531	162,577	162,577	283,036	213,249
	Tiempo (s)	3,71	3,86	3,84	3,74	3,90	3,81	3,82	3,89
(100) C101 SEED=24	FO	1226,69	1540,01	1817,22	1447,02	1292,84	1332,85	1489,56	1344,23
	N. Penaliz.	75	0	1	0	0	0	0	1
	P. Tiempo	57885,9	0	3,45906	0	0	0	0	0,80464
	Distancia	1226,69	1540,01	1815,49	1447,02	1292,84	1332,85	1489,56	1336,18
	Tiempo (s)	10,61	13,09	13,23	13,06	12,89	12,84	12,91	13,01
(200) C1.2.1 SEED=36	FO	3954,04	5830,37	6225,42	6384,64	5241,16	6145,43	7319,22	6004,93
	N. Penaliz.	155	13	7	6	1	0	1	1
	P. Tiempo	130070	422,539	218,416	199,08	62,1973	0	11,9016	5,33372
	Distancia	3954,04	5724,73	6116,21	6235,33	5178,96	6145,43	7259,71	5951,6
	Tiempo (s)	18,00	22,26	21,90	21,88	21,13	21,35	21,91	21,05
(400) C1.4.1 SEED=32	FO	12072,1	13972,8	17807,9	17033,3	20541,5	17433,4	19661	18570,1
	N. Penaliz.	308	31	21	15	18	3	1	0
	P. Tiempo	227800	1024,33	545,966	243,998	347,024	25,6175	10,3776	0
	Distancia	12072,1	13716,7	17534,9	16850,3	20194,4	17356,5	19609,1	18570,1
	Tiempo (s)	32,30	42,32	44,94	46,03	47,33	45,43	49,37	46,47
(1000) C110.1 SEED=42	FO	65513,9	90132,6	110302	114916	101571	121151	128280	129886
	N. Penaliz.	808	201	150	150	100	48	35	17
	P. Tiempo	751213	16925,8	8512,76	6491,03	3756,61	780,884	651,157	398,82
	Distancia	65513,9	85901,2	106046	110047	97814,6	118808	125024	125898
	Tiempo (s)	55,36	101,82	117,23	116,69	115,66	121,70	125,82	119,14

Nota: FO: Valor de la función objetivo. — N. Penaliz.: Cantidad de nodos penalizados por ventanas de tiempo. — P. Tiempo: Penalización total de tiempo acumulada. — Distancia: Distancia total recorrida por la flota.

El Cuadro 1 muestra que valores bajos de α saturan las restricciones temporales al escalar el problema, alcanzando un retraso de 751.213 s en la instancia de 1000 clientes. En contraposición, $\alpha = 1$ equilibra de mejor manera ambos objetivos; aunque mantiene 100 nodos penalizados en la instancia mayor debido a la estricta densidad del problema, minimiza drásticamente la magnitud del retraso acumulado y optimiza la distancia total, consolidando un valor de FO competitivo (101571). Por ello, se seleccionó $\alpha = 1$ para la calibración.

8.2. Calibración del Esquema de Enfriamiento y Mecanismo de Estancamiento

Tomando como base la instancia mediana de 400 clientes con un valor fijo de $\alpha = 1,0$, se procedió a aislar el impacto de la temperatura inicial (T_0), la tasa de enfriamiento (COOLING_RATE)

y el umbral de estancamiento (**MAX_STAGNATION**). Los resultados de estas tres fases consecutivas se consolidan en el Cuadro 2.

Cuadro 2: Resultados detallados del proceso secuencial de sintonización para la instancia de 400 clientes ($\alpha = 1,0$).

Parámetro Evaluado	Valor Objetivo	Nodos Penalizados	Penalización Tiempo	Distancia Recorrida
<i>Temperatura Inicial (T_0)</i>				
100	17088,6	11	183,465	16905,2
500	18108,8	11	253,658	17855,1
1000	18112,3	8	176,911	17935,4
5000	16022,8	11	82,1674	15940,6
10000	17445,3	13	196,747	17248,5
<i>Ratio de Enfriamiento</i>				
0.95	17321,4	15	103,058	17218,3
0.97	17360,2	13	184,298	17175,9
0.98	14271	15	219,675	14051,3
0.99	16022,8	11	82,1674	15940,6
0.995	17976,5	19	316,084	17660,4
<i>Umbral de Estancamiento</i>				
1000	26245,7	25	744,137	25501,5
2500	18621,3	20	411,262	18210
5000	14271	15	219,675	14051,3
10000	13410,7	7	41,2605	13369,4
15000	11965,3	3	33,4986	11931,8
20000	11175,8	1	12,1153	11163,6
30000	11175,8	1	12,1153	11163,6

El Cuadro 2 resume el proceso de calibración realizado sobre la instancia de 400 clientes.

La mejor temperatura inicial fue $T_0 = 5000$, mientras que una tasa de enfriamiento de 0,98 produjo el menor valor de función objetivo. Respecto al criterio de estancamiento, el desempeño mejoró progresivamente al aumentar **MAX_STAGNATION**, alcanzando su mejor resultado con 20000 iteraciones sin mejora. Valores superiores no generaron beneficios adicionales, por lo que dicha configuración fue seleccionada para la evaluación final.

8.3. Análisis de convergencia y robustez

Cuadro 3: Análisis de convergencia, calidad de resultados y robustez de la metaheurística.

Instancia	Mejor FO	Peor FO	Promedio FO	Desv. Estándar (σ)	Tiempo Prom. (s)
(25) C101	155,936	229,630	186,7418	39,2452	3,77
(100) C101	1174,650	1541,430	1280,2340	149,7322	12,34
(200) C1_2_1	3623,130	4910,680	4252,6580	474,2996	19,70
(400) C1_4_1	11270,400	13511,500	12282,9000	803,6600	32,15
(1000) C110_1	95470,300	101984,000	98508,0400	2510,3392	84,05

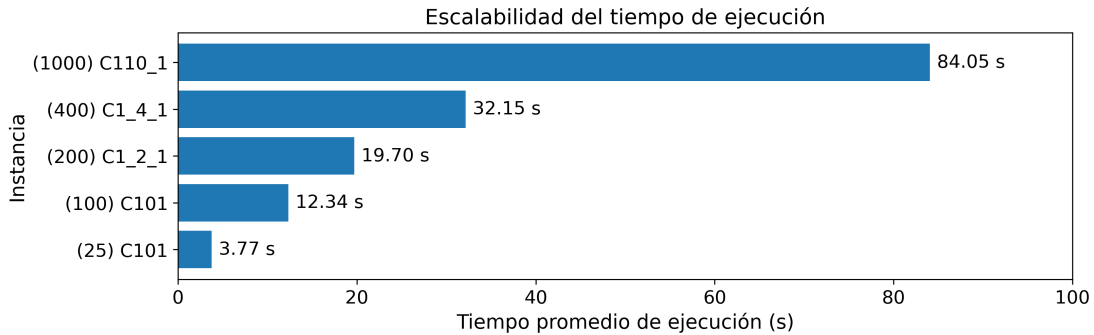


Figura 2: Escalabilidad del tiempo promedio de ejecución en segundos para cada instancia evaluada.

Para evaluar la robustez del algoritmo, cada instancia fue ejecutada cinco veces utilizando distintas semillas aleatorias. El Cuadro 3 presenta el mejor, peor y promedio valor de la función

objetivo, junto con la desviación estándar y el tiempo promedio de ejecución.

Los resultados muestran un incremento progresivo del tiempo computacional a medida que aumenta el tamaño de la instancia, pasando de 3,77 s para 25 clientes a 84,05 s para 1000 clientes. Este comportamiento es consistente con el crecimiento del espacio de búsqueda y evidencia una escalabilidad razonable de la metaheurística.

Respecto a la calidad de las soluciones, la diferencia entre el mejor y el peor valor de la función objetivo aumenta con el tamaño de la instancia, lo que indica una mayor sensibilidad al componente estocástico en problemas de mayor complejidad. Sin embargo, la desviación estándar permanece acotada en relación con la magnitud de la función objetivo obtenida en cada caso, sugiriendo un comportamiento estable entre ejecuciones.

En particular, para la instancia de 1000 clientes se obtiene un promedio de 98508,04 con una desviación estándar de 2510,34, equivalente a aproximadamente un 2,5 % del valor promedio. Este resultado indica que, pese al aumento del espacio de búsqueda, el algoritmo converge de manera consistente hacia soluciones de calidad similar.

En conjunto, los resultados muestran que la configuración seleccionada mantiene un equilibrio adecuado entre calidad de solución, robustez y tiempo computacional, permitiendo abordar instancias de hasta 1000 clientes con una variabilidad relativamente baja entre ejecuciones.

9. Conclusiones

El Multi-Depot Vehicle Routing Problem with Time Windows (MDVRP-TW) constituye un desafío de optimización combinatoria de alta complejidad. Su naturaleza NP-hard y el crecimiento exponencial del espacio de búsqueda justifican plenamente el uso de técnicas metaheurísticas frente a la inviabilidad operativa de los métodos exactos en escenarios de gran escala. En este trabajo, la implementación de Simulated Annealing (SA) con operadores de vecindad heterogéneos y una función objetivo penalizada demostró ser una propuesta metodológica adecuada, logrando resolver de forma sistemática las variantes de asignación, ruteo y cumplimiento de ventanas de tiempo asociadas al problema.

Desde el punto de vista experimental, las conclusiones se justifican directamente en los resultados cuantitativos consolidados. Esta técnica demostró una alta robustez al mantener una dispersión acotada entre ejecuciones estocásticas; incluso en la instancia más compleja de 1000 clientes, la desviación estándar se posicionó en un 2,5 % respecto al promedio (98508,04). Asimismo, el algoritmo exhibió una escalabilidad computacional sobresaliente y viable para entornos profesionales, requiriendo un promedio de apenas 3,77 segundos para la instancia menor de 25 clientes y logrando resolver la instancia crítica de 1000 clientes en un tiempo promedio de 84,05 segundos.

El proceso de ajuste de parámetros permitió aislar y validar el impacto crítico de los parámetros de control, condiciéndose plenamente con los objetivos de eficiencia planteados. Se determinó de forma empírica que el comportamiento del algoritmo alcanza su rendimiento óptimo bajo la configuración $\alpha = 1$, $T_0 = 5000$, $\text{COOLING_RATE} = 0,98$ y un umbral de $\text{MAX_STAGNATION} = 20000$ iteraciones. Los resultados contradijeron el supuesto inicial de que una penalización extrema de ventanas de tiempo ($\alpha \geq 3$) induciría soluciones más competitivas; por el contrario, se demostró que valores elevados de α vuelven restrictivo moverse entre vecindarios, inflando la distancia total y desoptimizando la función objetivo. En su lugar, el equilibrio de $\alpha = 1$ operó como configuración óptima que permite explorar zonas transitoriamente infactibles para alcanzar óptimos locales superiores.

En términos de ventajas frente al estado del arte, la propuesta destaca por su bajo costo computacional, robustez ante la aleatoriedad y simplicidad de implementación. No obstante, como desventaja intrínseca se reconoce su sensibilidad a la calibración inicial, puesto que un esquema de enfriamiento mal configurado degrada la convergencia. En comparación con enfoques heurísticos rígidos, el uso de múltiples vecindades dotó al algoritmo de una flexibilidad crucial

para escapar de óptimos locales. Como trabajo futuro, a partir de las limitaciones observadas en las instancias de mayor densidad, se considera relevante incorporar un mecanismo de control adaptativo que modifique dinámicamente los parámetros de control durante las iteraciones y explorar la hibridación con estrategias de búsqueda local intensiva.

10. Uso LLMs

10.1. Claude Sonnet 4.6

1. **Herramienta utilizada:** Claude Sonnet 4.6 (Anthropic), versión gratuita accesible en claude.ai.
2. **Propósito de uso:** Búsqueda bibliográfica para el estado del arte sobre MDVRP-TW, revisión del modelo matemático y generación de ideas iniciales.
3. **Prompts relevantes:** Solicitud de bibliografía, resúmenes y explicaciones sobre MDVRP-TW, y consultas sobre restricciones, variables y formulaciones típicas en modelos matemáticos de VRP.
4. **Nivel de edición:** Alto. La bibliografía y sugerencias matemáticas fueron seleccionadas, revisadas y ajustadas manualmente basándose en su relevancia técnica.
5. **Validación:** Se realizó una validación mediante la lectura directa de los artículos fuente, consultas en foros especializados (StackExchange) y análisis propio.

10.2. ChatGPT 5.5

1. **Herramienta utilizada:** ChatGPT 5.5 (OpenAI), versión gratuita accesible desde chatgpt.com.
2. **Propósito de uso:** Apoyo en la implementación de software en C++, incluyendo corrección de tipado, documentación, refactorización, estructuración del proyecto, creación del Makefile y debugging de errores. Además, revisión de redacción, ortografía y cumplimiento de formato.
3. **Prompts relevantes:** “¿Cómo puedo corregir este error de tipado?”, “refactoriza este bloque de código para mejorar la legibilidad”, “recomiendame como estructurar mi proyecto...”, “genera un Makefile para este proyecto” y “revisa este párrafo para mejorar su redacción académica”.
4. **Nivel de edición:** Alto. El código no fue copiado íntegramente; se utilizó para corregir problemas puntuales, mejorar la arquitectura y resolver bugs. Las sugerencias fueron revisadas y adaptadas manualmente.
5. **Validación:** El código fue validado por juicio propio, contrastando el funcionamiento con la lógica requerida y observando el código. La redacción fue verificada manualmente contra la rúbrica.

10.3. Gemini

1. **Herramienta utilizada:** Gemini (Google), versión gratuita.
2. **Propósito de uso:** Contrastar información con la bibliografía, mejorar la redacción académica, formatear tablas de Excel a LaTeX, explicar restricciones matemáticas y revisar el cumplimiento de la rúbrica.
3. **Prompts relevantes:** “Mejora la redacción académica de este párrafo”, “convierte estos datos a tabla LaTeX”, “explica esta restricción matemática”, “¿es correcta esta afirmación según el paper?” y “revisa si este texto cumple la rúbrica”.
4. **Nivel de edición:** Alto. Todo contenido fue revisado, editado y verificado manualmente para asegurar precisión técnica antes de su inclusión.
5. **Validación:** Se contrastó la información contra las fuentes originales, aplicando juicio propio para la verificación final y asegurando la alineación con los requisitos del informe.

Referencias

- [1] T. Barros-Everett, E. Montero y N. Rojas-Morales. «Parameter Prediction for Metaheuristic Algorithms Solving Routing Problems using Machine Learning». En: *Applied Sciences* 15.6 (2025). DOI: [10.3390/app15062946](https://doi.org/10.3390/app15062946).
- [2] J.-F. Cordeau, G. Laporte y A. Mercier. «A unified tabu search heuristic for vehicle routing problems with time windows». En: *Journal of the Operational Research Society* 52.8 (2001), págs. 928-936. URL: <http://www.jstor.org/stable/822953> (visitado 15-05-2026).
- [3] J.-F. Cordeau y M. Maischberger. «A parallel iterated tabu search heuristic for vehicle routing problems». En: *Computers & Operations Research* 39.9 (2012), págs. 2033-2050. DOI: <https://doi.org/10.1016/j.cor.2011.09.021>.
- [4] G. B. Dantzig y J. H. Ramser. «The Truck Dispatching Problem». En: *Management Science* 6.1 (1959), págs. 80-91. URL: <https://www.jstor.org/stable/2627477> (visitado 14-05-2026).
- [5] Z. Lei y J. Hao. «A GPU-Accelerated Hybrid Method for a Class of Multi-Depot Vehicle Routing Problems». En: *arXiv preprint* (2026). URL: <https://arxiv.org/abs/2605.05208> (visitado 14-05-2026).
- [6] X. Li et al. «A Two-Stage Optimization Approach for Multi-Depot Vehicle Routing Problem with Time Windows». En: *2024 4th International Conference on Computer Communication and Artificial Intelligence (CCAI)*. 2024, págs. 540-545. DOI: [10.1109/CCAI61966.2024.10603115](https://doi.org/10.1109/CCAI61966.2024.10603115).
- [7] W. Liu, Y. Luo e Y. Yu. «An Adaptive Hybrid Genetic and Large Neighborhood Search Approach for Multi-Attribute Vehicle Routing Problems». En: *arXiv preprint* (2024). URL: <https://arxiv.org/abs/2402.18903> (visitado 14-05-2026).
- [8] Z. Luo, H. Qin y A. Lim. «Branch-and-price-and-cut for the multiple traveling repairman problem with distance constraints». En: *Journal of Combinatorial Optimization* 234.1 (2014), págs. 49-60. DOI: <https://doi.org/10.1016/j.ejor.2013.09.014>.
- [9] M. Polacek et al. «A Variable Neighborhood Search for the Multi Depot Vehicle Routing Problem with Time Windows». En: *Journal of Heuristics* 10 (2004), págs. 613-627. DOI: <https://doi.org/10.1007/s10732-005-5432-5>.
- [10] M. E. H. Sadati, D. Aksen y B. Çatay. «An efficient variable neighborhood search with tabu shaking for a class of multi-depot vehicle routing problems». En: *Computers & Operations Research* 133 (2021), pág. 105269. DOI: <https://doi.org/10.1016/j.cor.2021.105269>.
- [11] L. Shen, F. Tao y S. Wang. «Multi-Depot Open Vehicle Routing Problem with Time Windows Based on Carbon Trading». En: *International Journal of Environmental Research and Public Health* 15.9 (2018). DOI: [10.3390/ijerph15092025](https://doi.org/10.3390/ijerph15092025).
- [12] M. M. Solomon. «Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints». En: *Operations Research* 35.2 (1987), págs. 254-265. URL: https://www-labs.iro.umontreal.ca/~dif6751/paper_solomon.pdf (visitado 14-05-2026).
- [13] K. C. Tan et al. «Heuristic methods for vehicle routing problem with time windows». En: *Artificial Intelligence in Engineering* 15.3 (2001), págs. 281-295. DOI: [https://doi.org/10.1016/S0954-1810\(01\)00005-X](https://doi.org/10.1016/S0954-1810(01)00005-X).
- [14] T. Vidal et al. «A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows». En: *Computers & Operations Research* 40.1 (2013), págs. 475-489. DOI: <https://doi.org/10.1016/j.cor.2012.07.018>.

- [15] T. Vidal et al. «Heuristics for multi-attribute vehicle routing problems: A survey and synthesis». En: *European Journal of Operational Research* 231.1 (2013), págs. 1-21. DOI: <https://doi.org/10.1016/j.ejor.2013.02.053>.
- [16] S. Voigt et al. «Hybrid adaptive large neighborhood search for vehicle routing problems with depot location decisions». En: *Computers & Operations Research* 146 (2022), pág. 105856. DOI: <https://doi.org/10.1016/j.cor.2022.105856>.
- [17] S. Wang, W. Sun y M. Huang. «An adaptive large neighborhood search for the multi-depot dynamic vehicle routing problem with time windows». En: *Computers & Industrial Engineering* 191 (2024), pág. 110122. DOI: <https://doi.org/10.1016/j.cie.2024.110122>.